Experiment 8 – Apply genetic algorithm on given problem.

Learning Objective: To understand and implement the Genetic Algorithm for solving optimization problems, and to analyse how evolutionary principles such as selection, crossover, and mutation can be applied to find optimal or near-optimal solutions using Python.

Tools: Visual Code Studio

Theory: -

Genetic Algorithms (GAs) are a class of evolutionary algorithms inspired by the process of natural selection and genetics in biological systems. They are widely used in Artificial Intelligence for solving optimization and search problems where traditional methods may be inefficient or infeasible. Genetic Algorithms work by evolving a population of candidate solutions over several generations to find the best or near-optimal solution.

In this experiment, the Genetic Algorithm is applied to solve a given optimization problem by simulating the principles of survival of the fittest.

1. Basic Concepts of Genetic Algorithm
A Genetic Algorithm operates on a population of individuals, where each individual represents a possible solution to the problem. These solutions are typically encoded as chromosomes (strings of genes), which can be represented using binary values, integers, or real numbers.

Key components of a Genetic Algorithm include:

- Population: A set of candidate solutions.
- Chromosome: Representation of a solution.
- Gene: A part of a chromosome representing a variable.
- Fitness Function: A function used to evaluate how good a solution is.

2. Working of Genetic Algorithm
The Genetic Algorithm follows an iterative process consisting of the following steps:

- Initialization:
  A random population of individuals is generated as the starting point.
- Fitness Evaluation:
  Each individual in the population is evaluated using the fitness function to determine its quality.
- Selection:
  Individuals with higher fitness values are selected for reproduction. Techniques such as roulette wheel selection or tournament selection are commonly used.
- Crossover (Recombination):
  Selected individuals (parents) are combined to produce offspring by exchanging parts of their chromosomes. This helps in exploring new regions of the solution space.
- Mutation:
  Small random changes are introduced in the offspring's genes to maintain diversity and avoid premature convergence.
- Replacement:
  The new generation replaces the old population, and the process repeats until a stopping condition is met (such as reaching a maximum number of generations or achieving a satisfactory fitness level).

3. Properties of Genetic Algorithm

- Works well for complex optimization problems.
- Does not require gradient information.
- Can escape local optima due to randomness.
- Provides approximate solutions rather than exact ones.

4. Advantages of Genetic Algorithm

- Suitable for large and complex search spaces.
- Flexible and adaptable to different types of problems.
- Can handle multi-objective optimization problems.

5. Limitations of Genetic Algorithm

- May require careful tuning of parameters (population size, mutation rate, etc.).
- Computationally expensive for large populations.
- No guarantee of finding the global optimum solution.

6. Applications of Genetic Algorithm
Genetic Algorithms are widely used in:

- Optimization problems (scheduling, routing, resource allocation)
- Machine learning and feature selection
- Engineering design and parameter tuning
- Game strategies and robotics

Overall, Genetic Algorithms provide a powerful and flexible approach for solving complex optimization problems by mimicking natural evolutionary processes. Understanding these concepts helps in developing intelligent systems capable of adaptive and efficient problem-solving.

<u>Code & Output</u>: -

Input: -

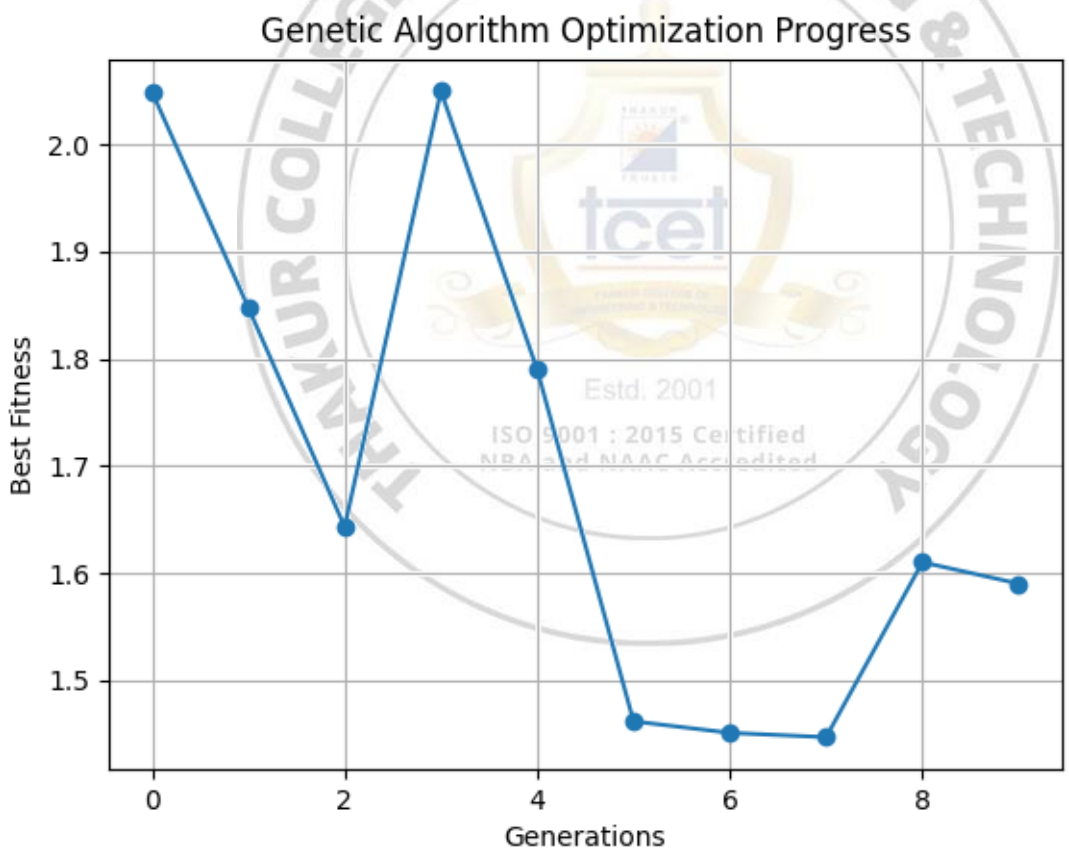
```
import numpy as np
import matplotlib.pyplot as plt

# -------- PARAMETERS (AS GIVEN) --------
POP_SIZE = 40
GENERATIONS = 10
X_MIN, X_MAX = -1.0, 2.0
CROSSOVER_PROB = 0.9
MUTATION_PROB = 0.2
MUTATION_STD = 0.1

np.random.seed(42)

# -------- FITNESS FUNCTION --------
# Maximize f(x) = x * sin(10πx) + 1
def fitness(x):
    return x * np.sin(10 * np.pi * x) + 1


# -------- INITIAL POPULATION --------
population = np.random.uniform(X_MIN,
X_MAX, POP_SIZE)


# -------- SELECTION (TOURNAMENT) --------
def selection(pop):
    i, j = np.random.randint(0, POP_SIZE, 2)
```

```
    return pop[i] if fitness(pop[i]) > fitness(pop[j])
else pop[j]


# -------- CROSSOVER --------
def crossover(p1, p2):
    if np.random.rand() < CROSSOVER_PROB:
        alpha = np.random.rand()
        child1 = alpha * p1 + (1 - alpha) * p2
        child2 = alpha * p2 + (1 - alpha) * p1
        return child1, child2
    return p1, p2


# -------- MUTATION --------
def mutate(x):
    if np.random.rand() < MUTATION_PROB:
        x = x + np.random.normal(0,
MUTATION_STD)
    return np.clip(x, X_MIN, X_MAX)


# -------- MAIN GA --------
best_fitness_progress = []

for gen in range(GENERATIONS):

    new_population = []

    for _ in range(POP_SIZE // 2):
        p1 = selection(population)
        p2 = selection(population)

        c1, c2 = crossover(p1, p2)
        c1 = mutate(c1)
        c2 = mutate(c2)

        new_population.extend([c1, c2])

    population = np.array(new_population)

    # Track best fitness
    fitness_values = fitness(population)
    best_fit = np.max(fitness_values)
    best_fitness_progress.append(best_fit)

    print(f"Generation {gen+1}: Best Fitness =
{best_fit:.4f}")

# -------- FINAL RESULT --------
best_index = np.argmax(fitness(population))
best_x = population[best_index]
best_y = fitness(best_x)

print("\nFinal Best Solution:")
print("x =", round(best_x, 4))
print("Fitness =", round(best_y, 4))


# -------- GRAPH --------
plt.plot(best_fitness_progress, marker='o')
plt.xlabel("Generations")
plt.ylabel("Best Fitness")
plt.title("Genetic Algorithm Optimization
Progress")
plt.grid()
plt.show()
```

Output: -

Generation 1: Best Fitness = 2.0489
Generation 2: Best Fitness = 1.8476
Generation 3: Best Fitness = 1.6431
Generation 4: Best Fitness = 2.0498
Generation 5: Best Fitness = 1.7897
Generation 6: Best Fitness = 1.4615
Generation 7: Best Fitness = 1.4508
Generation 8: Best Fitness = 1.4467
Generation 9: Best Fitness = 1.6100
Generation 10: Best Fitness = 1.5900

Final Best Solution:
x = 0.6653
Fitness = 1.59



Learning Outcomes: The student should have the ability to:

LO 8.1: Apply Genetic Algorithm techniques to solve optimization problems using population-based search methods.
LO 8.2: Understand and implement key operations such as selection, crossover, and mutation.
LO 8.3: Evaluate the performance of solutions using appropriate fitness functions.
LO 8.4: Analyse the effectiveness of evolutionary approaches in finding optimal or near-optimal solutions

Course Outcomes:

Upon completion of this experiment, students will be able to understand and apply Genetic Algorithms to solve optimization problems, and analyse the effectiveness of evolutionary techniques in developing intelligent and adaptive solutions in Artificial Intelligence.

Conclusion:

Jagdu Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				